

C++ Input Validation

Professional Techniques for Robust Programs

A comprehensive 11-page guide on securing your applications against invalid user data.

Introduction to Input Validation

Input validation is the process of ensuring that a program operates on clean, correct, and useful data. It is the first line of defence against software bugs, crashes, and security vulnerabilities.

In C++, user input is often unpredictable. Users might enter text when a number is expected, or enter values that are mathematically impossible for the program to process.

A robust program assumes that all external input is potentially 'bad' and checks it before allowing it to reach the core logic.

The Risks of Neglecting Validation

Without proper validation, several issues can occur:

1. Infinite Loops: If `cin` fails to read a value, the failed input stays in the buffer, causing subsequent reads to fail immediately.
2. Logic Errors: Processing a negative age or a zero divisor leads to incorrect results or runtime crashes.
3. Security Risks: Maliciously crafted inputs can be used to exploit buffer overflows or other vulnerabilities.
4. Poor User Experience: A program that crashes when a user makes a typo is frustrating and unprofessional.

Understanding cin.fail() and Error States

The standard input stream (`std::cin`) has internal state flags that track the success or failure of operations.

- `cin.fail()`: Returns true if an input operation failed (e.g., trying to read 'abc' into an int).
- `cin.eof()`: Returns true if the end of the file/stream is reached.
- `cin.bad()`: Returns true if a fatal system error occurred.
- `cin.good()`: Returns true if the stream is healthy and ready for operations.

The Recovery Process: Clear and Ignore

When `cin` enters a fail state, it stops working. To fix this, you must follow two steps:

1. `cin.clear()`: This resets the error flags so `cin` can be used again. However, the 'garbage' text is still sitting in the input buffer.
2. `cin.ignore()`: This removes the problematic characters from the buffer. A common usage is `cin.ignore(1000, '\n')`, which skips 1000 characters or until a newline is found.

Without these steps, your program will likely enter an infinite loop of 'Invalid Input' messages.

Numeric Range Validation

Even if the type is correct (e.g., the user entered an integer), the value might be logically invalid.

Range validation ensures the number fits within business rules (e.g., marks between 0-100, or a bank withdrawal amount that is less than the current balance).

Example logic: `if (value < MIN || value > MAX) { // handle error }`

This should be checked immediately after verifying that cin didn't fail.

Robust Loop Architectures

The best way to handle input is using a 'while' loop. This creates a trap that the user cannot leave until they provide valid data.

Syntax structure:

```
while (true) { cout << 'Prompt'; cin >> val; if (cin.fail()) { clear buffer; continue; }  
if (range invalid) { continue; } break; }
```

This pattern is the industry standard for console-based interactive applications.

Validating Strings and Empty Inputs

When using `std::string`, we often care about content rather than type. Using `getline(cin, name)` is preferred over `cin >> name` because it captures spaces.

Key checks for strings:

- `empty()`: Check if the user just pressed Enter without typing.
- `length()`: Ensure the input isn't too short (like a 2-character password) or too long.
- Character types: Looping through the string to check if it contains only alphabets or only digits.

Reusable Validation Functions

To keep `main()` clean, validation logic should be moved into functions. A function like `'int getValidInt(string prompt)'` can be called whenever a number is needed.

This follows the DRY (Don't Repeat Yourself) principle. By centralizing the logic, you only have to write the complex `cin.clear()` and `cin.ignore()` code once.

Validation functions make the program modular and much easier to debug.

Combining Exceptions with Validation

For more complex systems, you can 'throw' an exception when input is invalid.

This is useful when the code reading the input is deep inside a library or function, but the error needs to be reported to a higher level (like a UI manager).

```
try { val = getValidInput(); if(val < 0) throw runtime_error('Negative values not allowed'); }
```

This allows for a unified error-handling strategy across the whole application.

Character and Menu-Based Validation

In menu-driven programs, users often choose options (A, B, C or 1, 2, 3).

Validation here often involves a 'switch' statement with a 'default' case to handle invalid choices.

It is also important to handle case sensitivity (converting 'y' and 'Y' to a single standard) to ensure the user isn't punished for small typing variations.

Conclusion and Best Practices

Input validation is what separates a student assignment from a professional grade application.

Summary of Rules:

- Always check the state of cin.
- Always clear and ignore after a failure.
- Always provide clear feedback to the user on what they did wrong.
- Use loops to allow users to retry their input.
- Validate both the Type (int/string) and the Logic (range/content).